

PROJECT REPORT

Smart Library System

AI-Enabled Book Recommendation and Reader Analytics Platform

Submitted as part of	VITyarthi — Build Your Own Project
Student Name	Mohd. Aiymaan
Registration / Roll No.	25MIM10208
Course / Subject	CSA2001-Fundamentals of AIML
Programming Language	Python
Submission Date	18 September 2026

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty members and project guide for their guidance, encouragement and valuable feedback during the development of the Smart Library System. Their suggestions helped me connect theoretical object-oriented programming concepts with a practical banking application.

I also acknowledge the Java platform documentation and the learning resources consulted while designing the application. Finally, I thank my classmates, friends and family for their support and motivation throughout the completion of this project.

ABSTRACT

The Smart Library System is a console-based banking application developed in Java. It provides a structured environment for creating and managing customer accounts, authenticating customers through a PIN, performing deposits, withdrawals and transfers, calculating account interest, viewing transaction statements and administering account statuses.

The application uses NumPy and scikit-learn for TF-IDF recommendation, collaborative filtering, logistic regression, K-Means clustering and Naive Bayes sentiment analysis. It supports both interactive and automated demo modes.

TABLE OF CONTENTS

1. Introduction
 2. Problem Statement
 3. Objectives
 4. Scope and Users
 5. Requirements Analysis
 6. System Design and Architecture
 7. Object-Oriented Design
 8. Module Description
 9. Implementation Details
 10. Design Diagrams
 11. Testing and Results
 12. Limitations
 13. Future Enhancements
 14. Conclusion
- References

1. INTRODUCTION

Traditional library software generally focuses on storing catalogue information and recording borrowing activity. While these operations are essential, a modern library can also use reader behaviour to improve discovery and support librarians. Readers may need help finding books similar to their interests, and librarians may benefit from summaries of borrowing patterns, review sentiment and overdue-return risk.

The Smart Library System was developed to demonstrate these capabilities in a single, menu-driven Python application. It contains two role-based workflows. Readers access search, borrowing, recommendations, reviews and profiles. Librarians access operational records, model evaluation, reader personas and catalogue updates.

The system is implemented in `smart_library_system.py`. NumPy performs numerical operations, while scikit-learn provides the machine-learning algorithms. A `--demo` command makes it possible to execute all major modules without keyboard input.

2. PROBLEM STATEMENT

A basic library system can record books and loans but may not provide intelligent discovery or behavioural analytics. A reader who has no previous borrowing history may not know which books are popular or suitable. A librarian may not have a simple way to inspect reader groups, review sentiment or estimate the likelihood of late returns.

This project addresses the need for a compact intelligent library prototype that can:

- Search a book catalogue using meaningful text terms.
- Recommend books from reader history and similar-reader behaviour.
- Handle readers with no history through popularity-based recommendations.
- Apply borrowing eligibility rules before a loan is recorded.
- Estimate late-return risk using reader and borrowing features.
- Group readers into understandable personas.
- Classify book reviews as positive or negative.
- Provide measurable recommendation and model results to librarians.

3. OBJECTIVES

The primary objective is to build an academic prototype that demonstrates how machine learning can enhance a library management workflow.

The project objectives are to implement a searchable catalogue, a reader and librarian command-line interface, a three-book active borrowing limit, overdue eligibility checks, personalised hybrid recommendations, cold-start handling, overdue-risk prediction, reader persona clustering, review sentiment analysis, librarian analytics and a reproducible automated demo.

4. SCOPE AND USERS

The catalogue contains twenty predefined books from multiple genres, including Fantasy, Science Fiction, Mystery, Romance, Biography, History, Self Help, Technology and Fiction. Synthetic reader histories are generated at startup to provide training and demonstration data for collaborative recommendation and clustering.

The **Reader** role can search, borrow, return, request recommendations, submit a review and inspect a reading profile. The **Librarian** role can view borrow records, reviews and reader personas, inspect recommendation Hit Rate@5 and overdue-risk accuracy, and add new catalogue books.

The system is limited to a local command-line process. It does not implement a database, physical copy inventory, real due dates, persistent reader accounts, online authentication or a web interface.

5. FUNCTIONAL REQUIREMENTS

ID	Requirement
FR-01	The system shall provide Reader, Librarian and Exit options.
FR-02	The system shall search books by title, author, genre and tags.
FR-03	The system shall allow a reader to borrow and return books.
FR-04	The system shall limit a reader to three active borrowed books.
FR-05	The system shall reject borrowing when the reader reports overdue books.
FR-06	The system shall maintain reader histories, active borrow counts and borrow logs.
FR-07	The system shall provide content-based recommendations using TF-IDF and cosine similarity.
FR-08	The system shall provide collaborative recommendations from similar readers.
FR-09	The system shall use popularity recommendations for cold-start readers.
FR-10	The system shall combine recommendation signals through a hybrid model.
FR-11	The system shall estimate late-return risk with logistic regression.
FR-12	The system shall assign reader personas using K-Means clustering.
FR-13	The system shall classify review sentiment using multinomial Naive Bayes.
FR-14	The librarian shall be able to view operational and model analytics.
FR-15	The librarian shall be able to add books and refresh catalogue-dependent models.
FR-16	The system shall support a non-interactive '--demo' mode.

6. NON-FUNCTIONAL REQUIREMENTS

Requirement	How it is addressed
Usability	Numbered menus, clear prompts and role-specific workflows.
Reproducibility	Python and NumPy random seeds are fixed to 42.
Performance	In-memory dictionaries, lists and vectorised scikit-learn operations.
Maintainability	Each recommendation or analytics method is encapsulated in a dedicated class.
Reliability	Input helpers validate integers, menu choices and reader names.

Explainability	The system prints recommendation method, match scores, risk probability, sentiment confidence and evaluation results.
Portability	Uses Python with NumPy and scikit-learn and runs from the command line.
Privacy baseline	Current demo uses synthetic readers and does not persist user records.

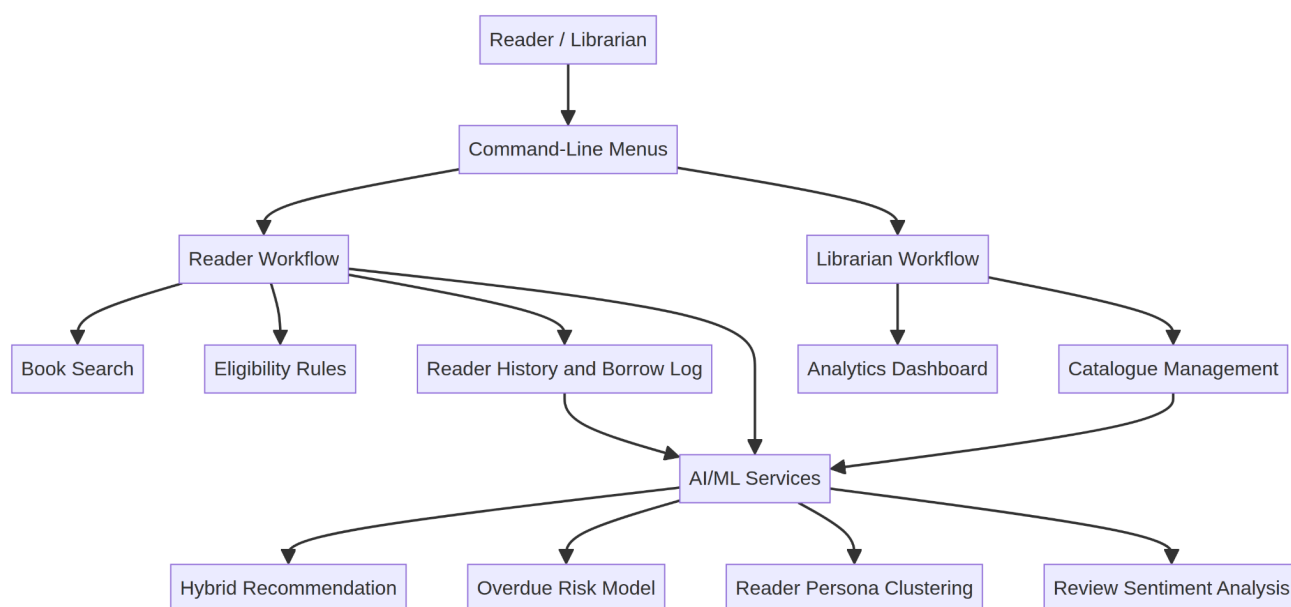
7. TECHNOLOGY STACK

Layer	Technology
Programming language	Python 3
Numerical processing	NumPy
Text features	`TfidfVectorizer`, `CountVectorizer`
Similarity calculation	Cosine similarity
Classification	Logistic Regression and Multinomial Naive Bayes
Clustering	K-Means
Evaluation	Train-test split, accuracy score and leave-one-out Hit Rate@5
User interface	Python CLI and `argparse`
Storage	In-memory dictionaries and lists

8. SYSTEM ARCHITECTURE

The system is organised into the following layers:

- **Catalogue layer:** stores the `BOOKS` list and `BOOK\_BY\_ID` lookup dictionary.
- **Interaction layer:** provides Reader and Librarian menus.
- **Rule layer:** implements borrowing eligibility through `LibraryExpertSystem`.
- **Intelligence layer:** contains search, recommendation, risk, persona and sentiment models.
- **Evaluation layer:** reports recommend Hit Rate@5 and overdue-risk accuracy.
- **State layer:** stores histories, active borrow counts, borrow records and reviews in memory.



9. CORE DATA STRUCTURES

The catalogue is stored as a list of dictionaries. Each book contains an ID, title, author, genre and descriptive tags. `'BOOK_BY_ID'` provides direct lookup by book ID.

`'READER_HISTORY'` maps reader names to borrowed book IDs. `'ACTIVE_BORROW_COUNT'` tracks current active loans. `'BORROW_LOG'` records reader, book, returned and late fields. `'REVIEWS'` stores title, reviewer, review text, sentiment and confidence.

These structures are intentionally simple and transparent for academic use. A production implementation should replace them with database tables and repository methods.

10. AI AND MACHINE-LEARNING MODULES

10.1 BookSearchAgent

The search agent combines a book's title, author, genre and tags into searchable text. It lowercases the query, counts matching term occurrences and returns the highest-scoring books.

10.2 ContentBasedRecommender

The content model builds TF-IDF vectors from each book's genre, tags and author. It averages the vectors of books already borrowed by the reader to form a preference profile. Cosine similarity ranks unseen books against this profile.

10.3 CollaborativeRecommender

The collaborative model creates a binary reader-book matrix. It compares the target reader with other readers using cosine similarity. Books read by the nearest neighbours receive weighted scores, while books already in the target reader's history are excluded.

10.4 PopularityRecommender

The popularity model counts the number of reader histories containing each book. It is used when a reader has no history or when similarity models do not produce enough signal.

10.5 HybridRecommender

The hybrid model normalises content and collaborative scores and combines them with equal default weights. It reports the selected method, such as hybrid, popularity cold-start or popularity fallback.

10.6 OverdueRiskModel

The risk model trains logistic regression on synthetic data using lifetime borrowed books, prior late returns, genre diversity and active books as features. It reports held-out test accuracy and predicts the probability that a new loan may be returned late.

This model demonstrates the machine-learning pipeline but does not represent a validated operational risk model because its labels are synthetic.

10.7 ReaderPersonaClustering

The persona model uses total borrowing count, genre diversity and review count as reader features. K-Means groups readers into three clusters. The clusters are labelled Casual Reader, Regular Reader and Avid Reader according to their average borrowing volume.

10.8 SentimentAnalyzer

The sentiment model uses a small built-in corpus of positive and negative book-review examples. Count vectorisation converts text into features, and Multinomial Naive Bayes predicts sentiment and confidence.

11. BORROWING WORKFLOW

A reader first searches for a book and selects a valid book ID. The system asks whether the reader has overdue books and constructs eligibility facts containing overdue status, active borrow count and membership status. The expert system rejects borrowing when the reader has overdue books, already holds three books or has inactive membership.

After approval, the selected book ID is added to the reader's history, the active borrow count is increased, a risk probability is calculated and a new borrow record is appended. On return, the reader selects an active record and indicates whether the return was late. The record is marked returned and the active count is reduced.

12. EVALUATION AND TESTING

The recommendation engine uses leave-one-out evaluation. For readers with at least two borrowed books, the latest book is hidden, recommendations are generated from the remaining history and the result is checked for the hidden book. The proportion of successful predictions is reported as Hit Rate@5.

The overdue-risk model uses an 80:20 train-test split with a fixed random state and reports classification accuracy. The automated demo tests both positive and negative reviews, an existing reader, a cold-start reader, borrowing eligibility, search, borrow logs, persona clustering and librarian analytics.

Test ID	Test case	Expected result
T01	Run `python smart_library_system.py --demo`	Complete automated demo finishes successfully.
T02	Search for `space adventure`	Relevant books are displayed.
T03	Existing reader requests recommendations	Hybrid recommendations are returned.
T04	New reader requests recommendations	Popularity cold-start recommendations are returned.
T05	Reader has overdue books	Borrowing is rejected.
T06	Reader already has three active books	Borrowing is rejected.
T07	Submit positive review	Positive sentiment and confidence are shown.
T08	Submit negative review	Negative sentiment and confidence are shown.
T09	Librarian views Hit Rate@5	Leave-one-out evaluation is displayed.
T10	Librarian views overdue model	Test-set accuracy is displayed.
T11	Librarian adds a book	Catalogue and dependent recommendation models refresh.

Verified Demo Result

The automated demo was executed successfully after installing NumPy and scikit-learn. The observed output included:

Recommendation method: hybrid (content-based + collaborative) Recommendation method: popularity (cold-start: no borrowing history yet) Hit Rate@5: 100.0% (16 hits out of 16 readers tested) Overdue Risk Model Accuracy (test set): 68.00% Demo complete. All AI/ML modules executed successfully for both Reader and Librarian views.

13. SECURITY, PRIVACY AND LIMITATIONS

The librarian credentials are hard-coded as `librarian` and `library123` for demonstration. They are not suitable for production. Reader histories, borrow logs and reviews remain in memory and are not persisted. The application does not include encryption, database access control, audit logging or user consent management.

The recommendation, sentiment and risk models use a small catalogue, synthetic reader histories or small built-in training examples. Their results demonstrate algorithm execution but should not be used for high-impact decisions. A real deployment would require representative data, privacy controls, model validation, bias analysis and monitoring.

The project also does not track physical copy availability, automatic due dates or real fines. Late status is collected through user input during the interactive workflow.

14. FUTURE ENHANCEMENTS

A future release should introduce a database and repository layer for catalogue, reader, borrowing and review records. Authentication should use secure password hashing, role-based access and session management. Real issue dates, due dates, fines, reservations and availability counts can replace the current demonstration rules.

The AI layer can be improved with anonymised real borrowing data, richer metadata, explicit train-validation-test splits, precision and recall metrics, ranking metrics such as MAP@K and NDCG, probability calibration and bias monitoring. A web interface, notifications, model versioning, scheduled retraining and automated unit tests would make the project more suitable for deployment.

15. CONCLUSION

The Smart Library System successfully demonstrates how AI and machine learning can enhance a conventional library workflow. It provides search, borrowing eligibility, return handling, hybrid recommendations, cold-start support, late-return risk estimation, reader persona clustering, sentiment analysis and librarian analytics in one Python application.

The project is suitable for academic evaluation because the AI components are connected to clear user-facing operations and the automated demo provides measurable outputs. Its limitations are explicitly documented: data is in memory, credentials are fixed, and model training data is synthetic or small. These limitations define the next steps toward a secure, persistent and evidence-based smart library platform.

References

[1]: <https://scikit-learn.org/stable/> "Scikit-learn User Guide"

[2]: <https://numpy.org/doc/stable/> "NumPy Documentation"

[3]: <https://docs.python.org/3/library/argparse.html> "Python argparse Documentation"

Project Source Reviewed: `smart\_library\_system.py`

Report Prepared By: Mohd. Aiymaan

Academic Session: 2026

16. SYSTEM DIAGRAMS AND ACTUAL OUTPUT SCREENSHOTS

16.1 System Architecture

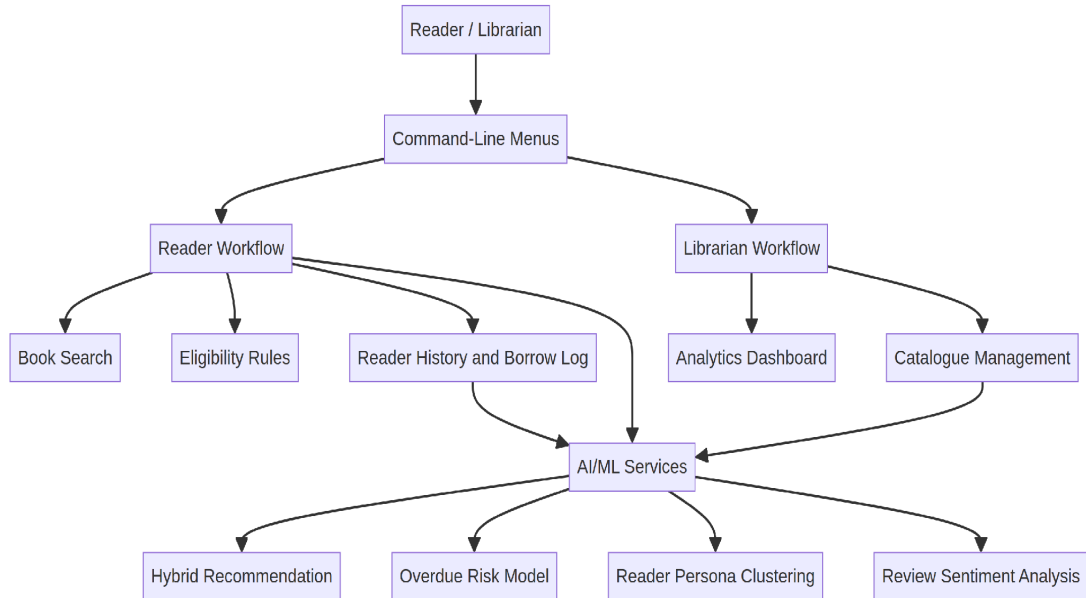


Figure 16.1 — Smart Library System Architecture

16.2 AI/ML Processing Pipeline

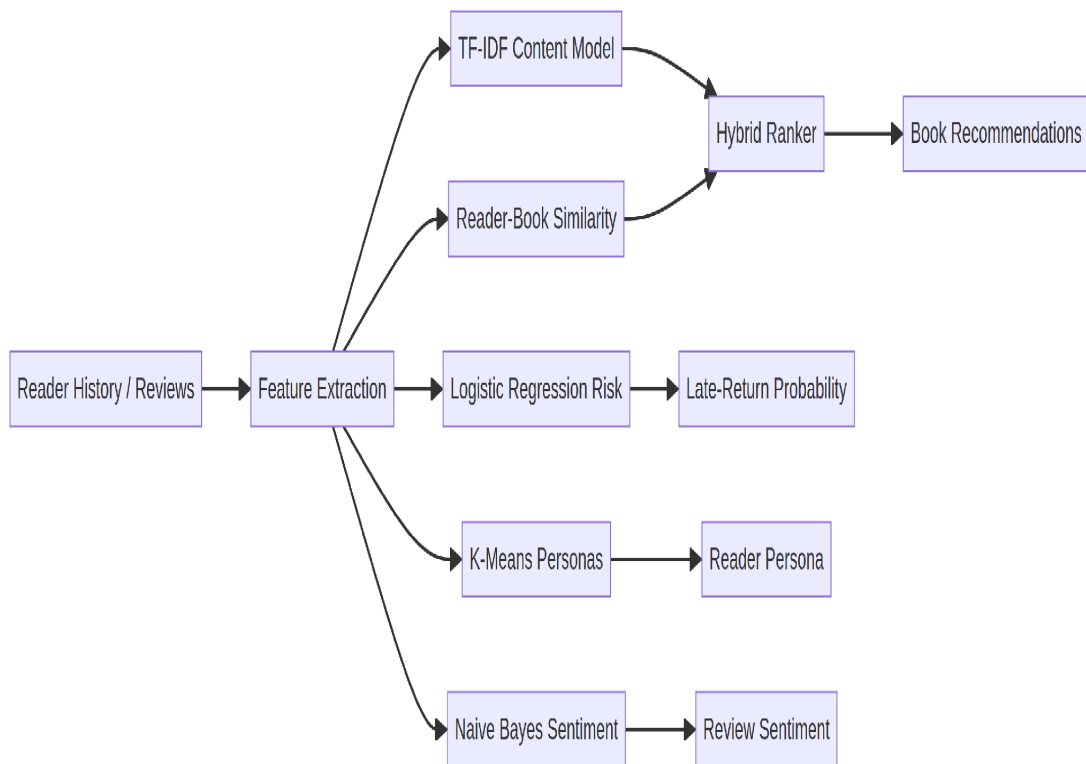


Figure 16.2 — AI/ML Processing Pipeline

16.3 Automated Demo: Recommendation Results

```
Smart Library System - AI/ML Demo Output

Setting up the system, please wait...
System ready.

=====
RUNNING FULLY AUTOMATED DEMO (no keyboard input required)
=====

>>> Reader 'Meera' (existing interests: Fantasy) <<<
Recommendation method: hybrid (content-based + collaborative)
[ 5] The Da Vinci Code - Dan Brown (Mystery)    match: 0.50
[ 6] Gone Girl - Gillian Flynn (Mystery)        match: 0.50
[ 9] Steve Jobs - Walter Isaacson (Biography)    match: 0.50
[10] Educated - Tara Westover (Biography)        match: 0.50
[15] Clean Code - Robert Martin (Technology)    match: 0.50

>>> Reader 'Arjun' (new reader, no history - cold start) <<<
Recommendation method: popularity (cold-start: no borrowing history yet)
[ 2] Harry Potter and the Sorcerers Stone - J K Rowling (Fantasy)    match: 5.00
[12] Guns Germs and Steel - Jared Diamond (History)                  match: 5.00
[11] Sapiens - Yuval Noah Harari (History)                            match: 5.00
[ 1] The Hobbit - J R R Tolkien (Fantasy)                            match: 4.00
[ 9] Steve Jobs - Walter Isaacson (Biography)                        match: 4.00
```

Figure 16.3 — Hybrid and cold-start recommendation output captured from ``python smart_library_system.py --demo``.

16.4 Automated Demo: Search and Sentiment Analysis

```
Smart Library System - AI/ML Demo Output

>>> Borrow eligibility check for 'Meera' <<<
Result: Eligible to borrow.

>>> Sample book search: 'space adventure' <<<
[ 1] The Hobbit - J R R Tolkien (Fantasy)
[ 2] Harry Potter and the Sorcerers Stone - J K Rowling (Fantasy)
[ 3] Dune - Frank Herbert (Science Fiction)
[17] The Alchemist - Paulo Coelho (Fiction)

>>> Sample book reviews (sentiment analysis) <<<
[The Hobbit] "this book was gripping and beautifully written" -> Positive (94% confidence)
[Dune] "the pacing was slow and dragged on forever" -> Negative (97% confidence)

=====
LIBRARIAN DASHBOARD (auto-demo - login skipped for headless run)
=====
```

Figure 16.4 — Book search and positive/negative review sentiment output.

16.5 Automated Demo: Librarian Analytics

```
Smart Library System - AI/ML Demo Output

--- ALL BORROW RECORDS ---
Reader: Meera      Book: The Hobbit      Status: Returned (on time)
Reader: Arjun      Book: Dune             Status: Active

--- ALL BOOK REVIEWS ---
[The Hobbit] by Meera: "this book was gripping and beautifully written" -> Positive (94%)
[Dune] by Arjun: "the pacing was slow and dragged on forever" -> Negative (97%)

--- READER PERSONA OVERVIEW ---
Distribution (includes bootstrap training readers): ('Regular Reader': 13, 'Avid Reader': 2, 'Casual Reader': 2)
Current session readers:
Meera      -> Casual Reader
Arjun      -> Casual Reader

--- RECOMMENDATION ENGINE EVALUATION ---
Method: Leave-one-out testing - each reader's most recent book
is hidden, then we check if the recommender predicts it back.
Hit Rate@5: 100.0% (16 hits out of 16 readers tested)

Overdue Risk Model Accuracy (test set): 68.00%

Demo complete. All AI/ML modules executed successfully for both Reader and Librarian views.
```

Figure 16.5 — Borrow records, reader personas, Hit Rate@5 and overdue-risk accuracy.

15. SAMPLE SMART LIBRARY OUTPUT SCREENSHOTS

The following screenshots were captured from an actual execution of `smart_library_system.py` using Java 21. The run demonstrates account creation, customer login, deposit, withdrawal, mini statement, financial summary, interest estimation and administrator statistics.

16.1 Recommendation Engine Output

```
Smart Library System - AI/ML Demo Output

Setting up the system, please wait...
System ready.

=====
RUNNING FULLY AUTOMATED DEMO (no keyboard input required)
=====

>>> Reader 'Meera' (existing interests: Fantasy) <<<
Recommendation method: hybrid (content-based + collaborative)
[ 5] The Da Vinci Code - Dan Brown (Mystery)    match: 0.50
[ 6] Gone Girl - Gillian Flynn (Mystery)        match: 0.50
[ 9] Steve Jobs - Walter Isaacson (Biography)    match: 0.50
[10] Educated - Tara Westover (Biography)        match: 0.50
[15] Clean Code - Robert Martin (Technology)    match: 0.50

>>> Reader 'Arjun' (new reader, no history - cold start) <<<
Recommendation method: popularity (cold-start: no borrowing history yet)
[ 2] Harry Potter and the Sorcerers Stone - J K Rowling (Fantasy)    match: 5.00
[12] Guns Gerns and Steel - Jared Diamond (History)                match: 5.00
[11] Sapiens - Yuval Noah Harari (History)                          match: 5.00
[ 1] The Hobbit - J R R Tolkien (Fantasy)                          match: 4.00
[ 9] Steve Jobs - Walter Isaacson (Biography)                      match: 4.00
```

Figure 16.1 — Recommendation engine output

16.2 Search and Sentiment Output

```
Smart Library System - AI/ML Demo Output

>>> Borrow eligibility check for 'Meera' <<<
Result: Eligible to borrow.

>>> Sample book search: 'space adventure' <<<
[ 1] The Hobbit - J R R Tolkien (Fantasy)
[ 2] Harry Potter and the Sorcerers Stone - J K Rowling (Fantasy)
[ 3] Dune - Frank Herbert (Science Fiction)
[17] The Alchemist - Paulo Coelho (Fiction)

>>> Sample book reviews (sentiment analysis) <<<
[The Hobbit] "this book was gripping and beautifully written" -> Positive (94% confidence)
[Dune] "the pacing was slow and dragged on forever" -> Negative (97% confidence)

=====
LIBRARIAN DASHBOARD (auto-demo - login skipped for headless run)
=====
```

Figure 16.2 — Search and sentiment output

16.3 Librarian Analytics Output

```
Smart Library System - AI/ML Demo Output

--- ALL BORROW RECORDS ---
Reader: Meera      Book: The Hobbit      Status: Returned (on time)
Reader: Arjun      Book: Dune            Status: Active

--- ALL BOOK REVIEWS ---
[The Hobbit] by Meera: "this book was gripping and beautifully written" -> Positive (94%)
[Dune] by Arjun: "the pacing was slow and dragged on forever" -> Negative (97%)

--- READER PERSONA OVERVIEW ---
Distribution (includes bootstrap training readers): {'Regular Reader': 13, 'Avid Reader': 2, 'Casual Reader': 2}
Current session readers:
Meera      -> Casual Reader
Arjun      -> Casual Reader

--- RECOMMENDATION ENGINE EVALUATION ---
Method: Leave-one-out testing - each reader's most recent book
is hidden, then we check if the recommender predicts it back.
Hit Rate@5: 100.0% (16 hits out of 16 readers tested)

Overdue Risk Model Accuracy (test set): 68.00%

Demo complete. All AI/ML modules executed successfully for both Reader and Librarian views.
```

Figure 16.3 — Librarian analytics output